

M14 / LEARNER BOOK

Kafka and streaming pipelines

Standalone Zero-to-Job-Ready Data Engineer 2026

12 controlled lessons

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

START HERE How to use this module	3
DE14-01 Batch vs streaming	4
DE14-02 Events, brokers and topics	8
DE14-03 Producers and message keys	12
DE14-04 Consumers and safe processing	16
DE14-05 Partitions and ordering	20
DE14-06 Offsets, group position and replay	23
DE14-07 Consumer groups and scaling	26
DE14-08 Delivery semantics and idempotency	29
DE14-09 Serialization, schemas and evolution	33
DE14-10 Python producer and consumer workflow	37
DE14-11 Spark Structured Streaming + Kafka	41
DE14-12 Lag, failures and streaming operations	45

START HERE

How to use this module

Open the matching practice folder from the course root: 13_M14/practice. Work on a copy. Commands printed in a lesson use that module as the working folder unless the lesson names a different location.

Study in lesson order. After the last lesson, use Quick Revision, take the Module Revision Test, then open its separate Protected Answers. Gates are additional assessments at the boundaries shown in the course guide.

Beginner-first event streaming: batch vs streaming through Kafka events/topics, producers/keys, consumers, partitions/order, offsets/replay, consumer groups, delivery semantics, schemas, Python clients, Spark Structured Streaming integration, and production lag/failure reasoning.

Course runtime baseline: Apache Kafka 4.3.1 (official supported release), confluent-kafka Python 2.15.0, and Apache Spark 4.2.0 for the Structured Streaming lesson. Live broker/client/Spark execution remains learner-PC proof unless explicitly recorded.

Complete M13. Keep Docker Desktop/WSL and other heavy services closed when they are not required. For the Kafka lab, the simplest current path is the official single-node Docker image. This is a learning environment, not a production-cluster design.

The official image starts a local Kafka environment with the current KRaft-era defaults. If port 9092 is already in use, stop the conflicting learner service rather than randomly changing Kafka settings. The practice README gives exact topic/CLI checks.

Practice: 13_M14/practice/ Quick revision:

12_REVISION_AND_TESTS/M14/M14_Quick_Revision_Book.pdf Protected answers (open only after saved attempt): 12_REVISION_AND_TESTS/M14/PROTECTED_OPEN_AFTER_ATTEMPT/M14_Module_Revision_Test_Answers.pdf

DE14-01

Batch vs streaming

Watch first

[1. Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:00-04:10: stream processing and batch/stream comparison

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

Learn streaming in small steps

First understand an event, its key, where it is stored and when a consumer records progress. Then run the small supplied fixture and one failure/replay exercise. Complete the M14 lessons and G05 requirements in order; you do not need to add production cluster administration, certification preparation or a multi-region Kafka deployment. Keep those as later specialization when your work needs them.

M14 lesson 1 of 12 | Primary practice: practice/kafka/01_* and simulator evidence where supplied

Why this matters

Most data engineers already understand batch because M08-M13 repeatedly used bounded files or tables. Streaming changes one assumption: there may be no natural 'all rows are here' moment. That affects when you compute, how you recover, and what 'done' means. Interviewers use this topic to see whether you can reason about continuous data rather than merely repeat that streaming is faster.

Picture it in your head

Picture a mandi truck register. A batch workflow is the clerk collecting all slips until 10 PM and then entering them together. A streaming workflow is each truck crossing the gate and creating an event immediately. The business can react sooner, but now the system must tolerate events arriving forever, late, duplicated, or temporarily out of order.

Words before we use them

Word	Meaning before we use it
event	One immutable statement that something happened, normally with a key, value and timestamp.
bounded data	A finite input with a known end, such as one daily file.
unbounded stream	A logical input that can continue producing events indefinitely.
latency	Time between an event happening and the useful result becoming available.
throughput	Amount of data processed per unit time.
micro-batch	A streaming engine execution style that processes small bounded chunks repeatedly.

Useful video / source route

[VIDEO: Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:00-04:10: stream processing and batch/stream comparison

[OFFICIAL/CURRENT: Official Spark 4.2 Structured Streaming + Kafka integration](#) Current Kafka source/sink, options and deployment package

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

A courier company receives parcel scans. In the batch design, 60,000 scans land as one hourly file. In the streaming design, each scan is emitted when the barcode is read. If fraud/routing alerts can wait an hour, batch may be simpler and cheaper. If a misrouted parcel must be stopped before the truck leaves, the value of low latency can justify Kafka and streaming complexity. The key engineering decision is not 'streaming is modern'; it is whether business latency, replay and continuous availability requirements justify the operational cost.

What you should see

You should be able to classify a workload by bounded/unbounded input, required response time, replay need and failure tolerance. You should also be able to say that Spark Structured Streaming usually executes continuous logic as repeated micro-batches even though the logical input is unbounded.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

A common beginner mistake is treating streaming as 'batch every second'. That misses persistent offsets, long-running consumer state, backpressure/lag and restart behavior. Another mistake is selecting Kafka because the source updates frequently even when the business only needs a daily result.

Reason through a realistic incident

An operations dashboard is 25 minutes behind. The producer is healthy and events are durable in Kafka. Is this data loss? Not necessarily. First separate event-time freshness from consumer progress: inspect the newest produced offset/timestamp, the consumer group's committed position and processing rate. Growing lag is a capacity or failure symptom, not proof that events disappeared.

Deepen the engineering reasoning

The important design move is to separate arrival cadence from decision cadence. Data may arrive every second while one consumer reacts instantly, another publishes five-minute operational summaries, and finance still closes a daily batch. Streaming therefore does not mean every calculation must run record-by-record forever. It means the system can observe an unbounded sequence and update work as new records arrive. Batch does not mean old-fashioned or inferior; a bounded input can be easier to reconcile, rerun and approve. When you design a pipeline, write the business latency target first: for example, alert within 10 seconds, dashboard within 5 minutes, settlement by 02:00. Then choose the simplest execution model that meets each target. Also name the recovery boundary. A continuously running job still needs durable source history or checkpoint/state evidence so a crash does not turn into silent data loss. The engineering goal is not 'real time everywhere'; it is correct enough, soon enough, with a recoverable history.

Try it yourself

For three cases - nightly payroll, parcel-location alerting, and a daily regulatory archive - choose batch, streaming, or a hybrid. For each, state the tolerated latency and the replay/recovery expectation.

Fresh changed retry

Change parcel alerting so the customer app may be 10 minutes late but the fraud engine must react in under 3 seconds. Explain why one event stream can feed consumers with different latency requirements.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying
- on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green

- process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the
- pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves
- mastery.

Revision in your own words

Streaming is not chosen because it is fashionable. Choose it when continuous arrival plus latency/replay requirements justify persistent consumer state and operational complexity.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-02

Events, brokers and topics

Watch first

[1. Confluent Developer - Topics | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:00-09:50: log abstraction, messages, timestamps, retention

[2. Confluent Developer - Brokers | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:00-03:12: brokers, clusters and KRaft

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 2 of 12 | Primary practice: practice/kafka/02_* and simulator evidence where supplied

Why this matters

Everything else in Kafka depends on a precise mental model of where an event goes. If you confuse a topic with a database table, or a broker with a consumer, later reasoning about partitions, retention and lag becomes fragile.

Picture it in your head

Think of a topic as a named, append-only stream of envelopes. Each broker is one warehouse machine that stores pieces of those streams. Producers deliver envelopes to the cluster; consumers read them without removing them. Kafka keeps events according to retention rules, so one consumer reading does not make the event vanish for another consumer.

Words before we use them

Word	Meaning before we use it
broker	A Kafka server that stores topic partitions and serves client requests.
cluster	A set of Kafka brokers operating as one logical service.
topic	A named append-only stream/log of events.
record/event	A key, value and metadata such as timestamp and headers.
retention	The rule controlling how long or how much log data Kafka keeps.
KRaft	Kafka's built-in metadata/controller architecture; the current route does not use ZooKeeper.

Useful video / source route

[VIDEO: Confluent Developer - Topics | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:00-09:50: log abstraction, messages, timestamps, retention

[VIDEO: Confluent Developer - Brokers | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:00-03:12: brokers, clusters and KRaft

[OFFICIAL/CURRENT: Official Apache Kafka Quickstart](#) Current 4.3.1 local/KRaft/Docker commands

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Start the official apache/kafka:4.3.1 container and create a topic called parcel-status. Produce three lines and consume them from the beginning. Then run a second consumer from the beginning. The same stored records are still available because consumption does not delete them. This is the first major difference from a simple work queue mental model.

```
docker exec m14-kafka /opt/kafka/bin/kafka-topics.sh --create --topic parcel-status --bootstrap-server localhost
docker exec -it m14-kafka /opt/kafka/bin/kafka-console-producer.sh --topic parcel-status --bootstrap-server localhost
docker exec -it m14-kafka /opt/kafka/bin/kafka-console-consumer.sh --topic parcel-status --from-beginning --bootstrap-server localhost
```

What you should see

A topic description should show at least one partition and leader. The console consumer should print the produced records. Re-running a consumer with an explicit from-beginning route can read retained records again.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

If the client cannot connect, distinguish 'container not running' from 'topic missing' from 'listener/port wrong'. Do not randomly change ports. First prove docker ps shows the broker, then list topics with the broker-local CLI, then test the host port.

Reason through a realistic incident

A team says yesterday's events are gone because their new consumer starts and sees only fresh traffic. Before blaming retention, check the consumer starting position and group state. A new group with auto.offset.reset=latest may begin at the end; that is different from the broker deleting history.

Deepen the engineering reasoning

A Kafka topic becomes easier to reason about when you stop picturing it as a mailbox that empties. Records are appended to partition logs and remain available according to retention policy. A consumer group stores its own progress separately, so two applications can read the same topic independently. That separation is why one stream can support fraud detection, analytics and auditing without copying the producer three times. Retention is therefore not cosmetic configuration: it sets a replay horizon. If the topic keeps seven days but your incident is discovered on day ten, the group offset alone cannot recover deleted records; you need another durable raw/archive source. Topic design also needs a business boundary. A topic named everything-events gives flexibility at first but often creates unclear ownership, schemas and retention. Prefer a topic whose event family, owners and contract can be stated. Finally, brokers host partitions and replicas, while the topic is the logical name. KRaft is the current Kafka coordination architecture in this course; ZooKeeper-era tutorials can still explain concepts, but they are not the setup authority.

Try it yourself

Create three domain topic names for an e-commerce system. For each, write one sentence describing what a single event means and what retention purpose the topic serves.

Fresh changed retry

Change the parcel-status topic from a transient dashboard feed to an audit source that must support seven-day replay. Explain which topic-level policy becomes more important and why consumers still need their own offsets.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Topics store retained events; brokers host partitions; consumers do not delete records when they read them. Kafka 4.x uses KRaft-era architecture, not ZooKeeper in this course.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-03

Producers and message keys

Watch first

[1. Confluent Developer - Partitions | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:41-03:23: scaling and partition ordering

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 3 of 12 | Primary practice: practice/kafka/03_* and simulator evidence where supplied

Why this matters

A producer is not merely 'send JSON'. Producer choices decide durability, partition placement and therefore ordering/scalability. Message keys are especially important because they connect business identity to partition-level ordering.

Picture it in your head

Imagine several parcel conveyor belts. The topic is the whole conveyor system; partitions are individual belts. A message key acts like a routing label. If all events for parcel P17 use the same stable key, the partitioner can keep those events on one belt, which makes per-parcel order tractable. If the key changes randomly, related events can scatter.

Words before we use them

Word	Meaning before we use it
producer	Client that publishes events to Kafka.
message key	Optional bytes used by partitioning logic and downstream semantics.
partitioner	Logic that chooses a partition when one is not explicitly supplied.
acks	Producer durability acknowledgement setting.
delivery callback	Function invoked when an asynchronous produce succeeds or permanently fails.
idempotent producer	Producer mode that prevents duplicates caused by retrying the same producer sequence under supported conditions.

Useful video / source route

[VIDEO: Confluent Developer - Kafka Producers](#) Optional visual - no verified bounded range; producer API, keys and delivery

[VIDEO: Confluent Developer - What are Kafka Producers and How do they work?](#) Optional visual - no verified bounded range; 5-minute producer overview

[VIDEO: Confluent Developer - Partitions | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:41-03:23: scaling and partition ordering

[OFFICIAL/CURRENT: Official Apache Kafka Producer Configs](#) Current acks/idempotence/retry behavior

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Publish shipment events keyed by shipment_id. Send SHP-42 CREATED, SHP-42 LOADED and SHP-42 DEPARTED using the same key, while other shipments use their own keys. Observe the delivery callback partition and offset. You are not trying to memorize a hash algorithm; you are proving that a stable key creates a stable routing intent and that delivery success must be observed rather than assumed when produce() returns.

What you should see

Each callback should identify a topic/partition/offset or an error. Repeated events with the same key should be routed consistently by a stable partitioner while the partition count is unchanged. Do not promise a partition number across a partition-count change.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Two dangerous beginner habits are ignoring delivery callbacks and using a random UUID as the key for every event. The first hides failed delivery; the second destroys useful grouping. Also do not hard-code a partition unless you own the operational consequences.

Reason through a realistic incident

A customer's status appears as DELIVERED before OUT_FOR_DELIVERY in a downstream service. Before blaming Kafka, ask whether both events had the same key and therefore the same partition. Kafka guarantees order within a partition, not global topic order across partitions.

Deepen the engineering reasoning

A key solves two related but different problems: routing and business grouping. If `order_id` is the key, all events for one order can be routed consistently while the partition count and partitioner behavior remain stable. That supports per-order ordering, but it does not mean every order in the topic is globally ordered relative to every other order. Write the sentence 'I need order within X' before choosing the key. Durability is a separate axis. The producer may accept a record into its local buffer before the broker has acknowledged it; delivery callbacks or equivalent delivery evidence tell you whether the record eventually reached the broker or failed. Current producer idempotence reduces some retry-duplicate risks, but it does not turn two separate calls carrying the same business event into one logical event. Your event contract still benefits from a stable `event_id`. In a design review ask: what key is used, what order does that promise, what happens if the key is hot, and what evidence proves delivery rather than merely enqueueing work locally?

Try it yourself

Design keys for order-events, customer-profile-changes and device-temperature. State what ordering you need and one hotspot/skew risk for each key.

Fresh changed retry

Assume one enterprise customer produces 40% of all events under one `customer_id` key. Keep per-order ordering while reducing hotspot risk by changing the key design to a finer business entity.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Use keys to express the unit whose order/grouping matters. Producer durability and delivery confirmation are separate from business-level deduplication downstream.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-04

Consumers and safe processing

Watch first

[1. Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 4 of 12 | Primary practice: practice/kafka/04_* and simulator evidence where supplied

Why this matters

Most streaming correctness bugs happen after an event is read: code performs a side effect, crashes, and then either loses or repeats work depending on when it committed progress. A safe consumer therefore needs a processing contract, not just a poll loop.

Picture it in your head

A consumer is a clerk reading numbered pages from a durable log. The clerk can bookmark progress with an offset commit. If the bookmark is moved before the page is actually processed, a crash can skip work. If the bookmark moves after durable work, a crash can repeat a page - so the side effect must tolerate replay.

Words before we use them

Word	Meaning before we use it
poll	Ask the broker/client library for available records and background events.
commit	Persist a consumer group position for a partition.
auto commit	Client-managed periodic offset commit; convenient but easy to misunderstand.
manual commit	Application-controlled commit after a chosen processing boundary.
side effect	External durable action such as database write, file append or API call.
poison record	Record that repeatedly fails validation/processing and can block progress without a policy.

Useful video / source route

[VIDEO: Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

[VIDEO: Confluent Developer - What is a Kafka Consumer and How does it work?](#) Optional visual - no verified bounded range; 6-minute consumer overview

[OFFICIAL/CURRENT: Official Apache Kafka Consumer Configs](#) Current group/offset/commit client configuration

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Use a consumer with `enable.auto.commit=False`. For each parcel event: decode JSON, validate required fields, write/upsert to a local durable sink using `event_id` as a uniqueness key, then synchronously commit that message's offset. If validation fails, save the raw event and reason to a quarantine file instead of silently committing as success. The exact production retry/DLQ architecture can vary, but the ordering of 'durable outcome before progress commit' is the important reasoning pattern.

What you should see

After a clean run, the sink contains one durable row per `event_id` and the group has a committed position beyond processed records. If the process is killed after the sink write but before the commit, replay may happen; the idempotent sink prevents a duplicate business effect.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Committing first is the classic loss window. Writing an external API/database and assuming Kafka transactions cover that external side effect is another mistake. Kafka's own transactional guarantees do not magically make arbitrary third-party systems transactional.

Reason through a realistic incident

A payment consumer restarted and sent one email twice. The event existed once in Kafka. That can still happen: the email side effect succeeded, the process died before progress was safely recorded, and the event replayed. Fix the side-effect contract or deduplication, not the event log.

Deepen the engineering reasoning

The safest consumer design names a handled outcome for every record. A valid record may create or upsert a durable business row. An invalid record may be durably quarantined with the raw bytes, topic, partition, offset and failure reason. Only after one of those outcomes is durable should the application advance committed progress under the chosen policy. That is why the supplied consumer persists quarantine evidence before committing a poison record: logging 'bad record' to the console is not durable handling. This still does not create universal exactly-once behavior. If the valid path calls an external payment or email API, a crash can occur after the remote side effect succeeds but before the Kafka progress commit. Replaying the event is correct from Kafka's perspective, but the external effect may repeat unless it accepts an idempotency key or you coordinate work through a durable outbox/state table. The useful mental model is: Kafka controls read progress; your application owns the side-effect contract. Test the crash window deliberately rather than assuming a happy-path run proves correctness.

Try it yourself

Write the safe order of operations for a consumer that writes to PostgreSQL. Include validation, durable write, reconciliation evidence and offset commit.

Fresh changed retry

Change the sink to a non-idempotent REST API. Explain why the same offset strategy is no longer sufficient and propose an outbox/idempotency-key or durable work-state approach.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Consumer correctness is the relationship between processing, durable side effects and committed offsets. Prefer replay-safe work over pretending replay never happens.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

Trace the commit decision

For malformed input, preserve the raw event and a rejection reason durably, then commit that Kafka message. For a valid event, write the business ledger durably, then commit. If the valid write fails, propagate the error and do not commit or relabel it as bad source data. A commit failure after a successful ledger write can cause replay. Identical event_id and payload then becomes a no-op; changed payload under that ID is a conflict requiring investigation.

DE14-05

Partitions and ordering

Watch first

[1. Confluent Developer - Partitions | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:41-03:23: scaling and partition ordering

[2. Confluent Developer - Topics | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:00-09:50: log abstraction, messages, timestamps, retention

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 5 of 12 | Primary practice: practice/kafka/05_* and simulator evidence where supplied

Why this matters

Partitions give Kafka parallelism and scale, but they also define the boundary of ordering. This trade-off appears constantly in interviews and production incidents.

Picture it in your head

A topic with four partitions is four ordered notebooks, not one globally ordered notebook. Each notebook has its own offsets 0,1,2... Reading all notebooks together can interleave events in many valid ways. The only strict order you can rely on is inside one partition.

Words before we use them

Word	Meaning before we use it
partition	Ordered append-only sub-log within a topic.
partition key	Business key whose records should route consistently.
global order	One total order across all events - Kafka partitions do not provide this by default.
hot partition	Partition receiving disproportionate load.
skew	Uneven distribution of events/work across partitions.
partition count	Number of parallel sub-logs; changing it can change future key routing.

Useful video / source route

[VIDEO: Confluent Developer - Partitions | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:41-03:23: scaling and partition ordering

[VIDEO: Confluent Developer - Topics | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:00-09:50: log abstraction, messages, timestamps, retention

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Take 12 events for four shipments. Route each shipment consistently to one simulated partition, then print offsets per partition. You will see order preserved for SHP-1 inside its partition even though events from SHP-1 and SHP-2 interleave when viewed across the topic. Next simulate one key producing eight of the twelve events and observe hotspot risk.

What you should see

You should be able to reconstruct each business entity's local order from its partition sequence. You should not claim the combined topic has a single meaningful order unless the topic has one partition or the application builds another ordering mechanism.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Increasing partitions after designing 'hash(key) -> partition' can move future records for the same key, depending on the partitioner. Also, choosing customer_id as a key may preserve customer order but create a hotspot for a huge customer. Key design is a business and distribution decision together.

Reason through a realistic incident

Only partition 2 has rising lag while partitions 0,1,3 are healthy. This is a clue: inspect key distribution and slow records for that partition. Scaling consumers cannot overcome a single hot partition if only one consumer can actively process that partition in the group at a time.

Deepen the engineering reasoning

Partition count is both a scalability choice and a correctness boundary. A consumer group can process different partitions in parallel, while each partition remains ordered. Six partitions can therefore provide up to six active partition assignments in a group at one moment, but useful throughput is still constrained by record cost, broker capacity, downstream sinks and skew. More partitions are not free: they add metadata, open files, leader/replica work and more ordering boundaries. Key distribution matters just as much as count. A single hot key stays on one partition, so adding consumers can leave the bottleneck unchanged while other consumers sit idle. When a dashboard shows events 'out of order,' first compare topic, partition and offset for the related business entity. If the events are in different partitions, Kafka may have preserved every local log perfectly while the merged display interleaves them. If strict global order is truly required, one partition is the simplest boundary but sacrifices parallelism. Most real systems narrow the required order to an entity such as order_id, device_id or account_id.

Try it yourself

For a food-delivery stream choose a key for driver-location, restaurant-order and payment-event topics. State exactly which order each choice preserves.

Fresh changed retry

A marketplace has one seller generating 50% of orders. Redesign a seller_id-keyed topic so order-level processing scales while seller-level totals can still be computed downstream.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Partitions are the unit of parallelism and ordered log storage. Preserve the order that matters by keying related events together, then watch for skew.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-06

Offsets, group position and replay

Watch first

[1. Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 6 of 12 | Primary practice: practice/kafka/06_* and simulator evidence where supplied

Why this matters

Offsets make Kafka replayable. They let a consumer group say 'this is how far I have safely progressed' without deleting the underlying events. Understanding offsets is what separates Kafka from a simple queue mental model.

Picture it in your head

Each partition is a book with numbered lines. The offset is the line number, not a business ID. A consumer group's committed offset is a bookmark. Resetting or seeking changes the bookmark; it does not rewrite the book.

Words before we use them

Word	Meaning before we use it
offset	Monotonic position of a record within one partition.
committed offset	Stored group progress used after restart/reassignment.
earliest/latest	Common reset policies when a valid committed position is absent.
replay	Read retained events again from an earlier position.
seek/reset	Explicitly change the position from which a consumer reads.
retention boundary	Old data may no longer exist if the topic has already deleted it.

Useful video / source route

[VIDEO: Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

[OFFICIAL/CURRENT: Official Kafka distribution - consumer offset tracking](#) How committed offsets are stored and recovered

[OFFICIAL/CURRENT: Official Apache Kafka Consumer Configs](#) Current group/offset/commit client configuration

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Consume five events and commit after event 5. Produce three more. Restart the same group: it should resume after the committed position. Then use a new group or explicit reset route to read earlier retained events. This proves that group position and topic storage are separate concepts.

What you should see

Same group + valid commit resumes from its recorded positions. New group behavior depends on reset policy. Replay works only while the required offsets still exist under retention.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

`auto.offset.reset` does not normally override an existing valid committed offset. Another common mistake is saying `'offset 50'` without naming the partition; offsets are partition-local.

Reason through a realistic incident

A bug corrupted derived rows between noon and 12:20. You fix the code and want to replay. First identify the source partitions and time/offset boundary, preserve existing evidence, and ensure the sink is idempotent or is rebuilt in an isolated target. Blindly rewinding a production group can duplicate external effects.

Deepen the engineering reasoning

Offsets are mechanical positions, not business timestamps or IDs. Partition 0 offset 25 and partition 1 offset 25 are unrelated positions in different logs. A consumer group's committed offsets describe restart progress, while the broker's log-end offsets describe how far each partition has advanced. Their difference is one form of lag. Replay is therefore a controlled change in read position, not a mutation of Kafka history. A common beginner mistake is setting `auto.offset.reset=earliest` on an existing group and expecting it to reread everything. That policy is consulted when the group has no valid committed position for a partition; an existing valid commit normally wins. A safe replay plan names the exact partitions and range, retention availability, destination/sink isolation, idempotency strategy and reconciliation after replay. For production repair, it is often safer to replay into an isolated corrected target or with a new group/application identity than to rewind a live consumer blindly. Preserve the original evidence so you can prove what was reread and what changed.

Try it yourself

Describe a safe replay plan for rebuilding yesterday's shipping aggregate from retained Kafka events into a new table.

Fresh changed retry

Now assume retention is only 6 hours and the bug happened yesterday. Explain why Kafka alone can no longer replay that window and which upstream/raw archive design from earlier modules would save you.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Offsets are partition positions; commits are group progress; replay changes reading position while retained log data stays immutable.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-07

Consumer groups and scaling

Watch first

[1. Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 7 of 12 | Primary practice: practice/kafka/07_* and simulator evidence where supplied

Why this matters

Consumer groups are how Kafka parallelizes one logical application while allowing completely separate applications to read the same topic independently. Misunderstanding this leads to both under-scaling and duplicate business work.

Picture it in your head

Imagine six partitions and three workers wearing the same team badge (group.id). Kafka assigns partitions among them, so each partition has one active worker in that group. A second team with a different badge can read the same six partitions independently.

Words before we use them

Word	Meaning before we use it
group.id	Name identifying one logical consumer application/group.
assignment	Mapping of topic partitions to group members.
rebalance	Redistribution of assignments when membership/topic state changes.
group coordinator	Broker-side component coordinating group membership and progress.
parallelism ceiling	For normal consumer groups, useful active consumers for one topic are limited by partition availability.
consumer protocol	Modern Kafka group protocol available in Kafka 4.x; classic groups also exist.

Useful video / source route

[VIDEO: Confluent Developer - Consumers | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters: 01:13-05:04 subscription/records/log; 05:04-06:31 commits; 06:31-08:04 groups/rebalancing

[VIDEO: Confluent Developer - Consumer Group Protocol](#) Optional visual - no verified bounded range; deeper coordinator/rebalance explanation

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Model a topic with six partitions. With one consumer, it owns all six. With two, assignments split. With six, each can own one. With eight, two have no partition work for that topic. Then create analytics-v1 and email-v1 as different group IDs: each group receives the full topic independently.

What you should see

You should be able to predict whether adding consumers increases parallelism and whether two consumers will share work or each receive all events based on group.id.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Starting every replica with a random group.id creates independent readers and can multiply side effects. Starting unrelated applications with the same group.id makes them accidentally share work. Both bugs look like 'Kafka randomly lost/duplicated messages' until group identity is inspected.

Reason through a realistic incident

A deployment doubles replicas from 3 to 8, but throughput barely changes on a 3-partition topic. That is expected: normal group parallelism is partition-bounded. Either processing per partition must get faster or partitioning must be reconsidered with key/order consequences.

Deepen the engineering reasoning

A consumer group is an application-level collaboration boundary. Consumers with the same group.id divide the group's assigned partitions; consumers with different group IDs are independent subscriptions and may each process every retained record. That distinction is how one topic can feed both fraud and analytics. Scaling a group depends on partition count: if a topic has six partitions, ten healthy consumers in one group cannot all own a partition simultaneously in the ordinary model. Four may remain idle, though they can help after failures or rebalances. Rebalancing is not merely a harmless control-plane detail. If consumers join or leave repeatedly, or processing blocks polling/heartbeats, partitions may move and work may pause. Records near the handoff can replay depending on commit timing. An incident timeline should therefore record assignment changes, poll/processing latency and commit positions. Before 'add more consumers,' determine whether the bottleneck is lack of parallel partitions, one hot partition, slow downstream I/O, excessive rebalance churn or expensive record processing. Scaling the wrong layer adds complexity without fixing throughput.

Try it yourself

Given 12 partitions, design consumer counts for email, fraud and analytics applications. Choose group IDs and explain which apps share vs independently read records.

Fresh changed retry

Fraud needs 24 workers but must preserve per-card order. Explain why simply adding workers is insufficient if there are only 12 partitions, and what trade-offs exist in repartitioning/key design.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Same group shares partition work; different groups independently consume the same retained stream. Scaling is constrained by partitions and processing cost.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-08

Delivery semantics and idempotency

Watch first

[1. Confluent - Apache Kafka Transactions: Message Delivery and Exactly-Once Semantics](#)

Optional visual - no verified bounded range; Jun Rao explains transactions and caveats

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 8 of 12 | Primary practice: `practice/kafka/08_*` and simulator evidence where supplied

Why this matters

'Exactly once' is one of the most abused phrases in data engineering. You need to separate producer duplicate prevention, Kafka transactions, consumer offset behavior and external side effects. Correct answers state the boundary of the guarantee.

Picture it in your head

Think of a bank transfer request crossing several checkpoints. Preventing the Kafka producer from appending the same retry twice is one boundary. Atomically writing Kafka output and Kafka offsets is another. Calling a bank API or inserting into a separate database is outside that Kafka transaction unless you design an additional idempotency/transaction pattern.

Words before we use them

Word	Meaning before we use it
at-most-once	Processing strategy that can avoid duplicates but may lose work on failure.
at-least-once	Processing strategy that avoids loss but may repeat work after failure.
idempotent operation	Repeating the same logical operation does not change the final state after the first successful application.
producer idempotence	Kafka producer/broker mechanism that deduplicates retried producer sequences under supported configuration.
transaction	Atomic unit allowing related Kafka writes and offset operations to commit/abort together.
effectively once	End result behaves once because replay is contained by transaction/idempotent state design.

Useful video / source route

[OFFICIAL/CURRENT: Official Apache Kafka Producer Configs](#) Current acks/idempotence/retry behavior

[VIDEO: Confluent - Apache Kafka Transactions: Message Delivery and Exactly-Once Semantics](#)

Optional visual - no verified bounded range; Jun Rao explains transactions and caveats

[VIDEO: Confluent Developer - Delivery Guarantees & Transactions](#) Optional visual - no verified bounded range; 7-minute at-most/at-least/effectively-once overview

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Simulate an order-status consumer. Run 1 writes order O9=PAID, crashes before its progress commit, and run 2 replays the same event. An append-only sink now has two rows; an upsert keyed by event_id or order/version has one final effect. Then contrast this with Kafka producer idempotence: it protects producer retry duplicates, but it does not automatically deduplicate two intentionally produced business events with different identities.

What you should see

You should be able to draw a boundary around what Kafka's idempotent producer/transactions protect and what still requires application or sink idempotency. Current Kafka producer idempotence is enabled by default when no conflicting configs disable it; acks=all is part of the compatible setting set.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Saying 'Kafka guarantees exactly once end to end' without naming source, processor and sink is unsafe. Another mistake is using payload equality as deduplication identity; two legitimate identical-valued events can exist.

Reason through a realistic incident

A consumer writes to a payment database and then commits its offset. It crashes after the DB commit. On restart the event is replayed. A Kafka transaction cannot roll back the external DB commit. Use a business idempotency key, transactional outbox/inbox, or equivalent durable coordination pattern.

Deepen the engineering reasoning

Delivery guarantees must always name their boundary. Producer idempotence is about preventing duplicate records caused by certain producer retry sequences; it is not a business deduplication service for two intentionally submitted events. Kafka transactions can atomically coordinate Kafka records and consumed offsets for supported Kafka-to-Kafka workflows, but an external database, email provider or payment API is outside that transaction unless you add another coordination mechanism. This is why 'exactly once' is incomplete by itself. Ask: exactly once where, and for which effect? For an external sink, a stable event_id plus a uniqueness constraint/upsert, an idempotency key accepted by the API, or a transactional outbox/inbox can convert replay into a harmless repeat. Separate duplicate records from duplicate business effects. A topic might contain the same event twice yet an idempotent sink applies one effect; conversely one Kafka record can trigger two emails if application logic retries incorrectly. Evidence should therefore include both transport metadata and sink/business reconciliation.

Try it yourself

For a Kafka -> Python -> PostgreSQL pipeline, state the intended delivery semantics and the exact mechanism preventing duplicate business effects.

Fresh changed retry

Change PostgreSQL to an external email API that offers an Idempotency-Key header. Design the event_id/idempotency-key mapping and evidence you would save.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green
- process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the
- pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves
- mastery.

Revision in your own words

Guarantees are scoped. Producer idempotence handles producer retries; replay-safe consumers/sinks handle external effects; Kafka transactions cover Kafka-managed atomic boundaries, not arbitrary external systems.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-09

Serialization, schemas and evolution

Watch first

[1. Confluent Developer - Confluent Schema Registry | Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:51-07:41: need, write/read, formats, compatibility motivation

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 9 of 12 | Primary practice: practice/kafka/09_* and simulator evidence where supplied

Why this matters

Kafka stores keys and values as bytes. Teams still need a contract that says what those bytes mean. Without schema discipline, a producer can make a harmless-looking field change that breaks many consumers at once.

Picture it in your head

A schema is the label and shape printed on a shipping form. Serialization turns the structured form into bytes for transport; deserialization reconstructs it. Schema evolution is changing the form while old and new readers may coexist.

Words before we use them

Word	Meaning before we use it
serialization	Convert an in-memory structure to bytes.
deserialization	Convert bytes back into a usable structure.
schema	Machine-readable contract for fields/types/structure.
schema registry	Service that stores schema versions/subjects and can enforce compatibility policies.
backward compatibility	A newer reader can read data written with an older schema, under the format's rules.
forward compatibility	An older reader can read data written with a newer schema, under the format's rules.

Useful video / source route

[VIDEO: Confluent Developer - Confluent Schema Registry | Kafka 101 \(2025 Edition\)](#) Verified chapters 00:51-07:41: need, write/read, formats, compatibility motivation

[VIDEO: Confluent Developer - Schema Registry course page](#) Optional visual - no verified bounded range; schema contracts and evolution

[OFFICIAL/CURRENT: Confluent Python client API 2.15.0](#) Current Producer/Consumer Python API reference

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Start with order schema v1: order_id string, amount number. Evolve to v2 by adding optional currency with a safe default. A v1 event can still be interpreted by a reader that understands v2. Contrast that with renaming order_id to id without a compatibility strategy: consumers expecting order_id break even though the JSON still looks valid.

What you should see

You should be able to classify additive optional changes as generally safer than destructive rename/type changes, while acknowledging exact compatibility rules depend on Avro/Protobuf/JSON Schema and registry configuration.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

'JSON is self-describing so we do not need schemas' is a weak production argument. JSON syntax tells you that a field exists, not whether a business contract permits it, whether it is required, or whether changing its type is compatible.

Reason through a realistic incident

A producer changes amount from numeric 19.99 to string '19.99 INR'. Some consumers silently cast; others fail. The real problem is contract evolution without controlled compatibility. Restore/introduce a schema boundary and change representation through an explicit versioned migration.

Deepen the engineering reasoning

Serialization answers 'how are these bytes interpreted?' while a schema answers 'what fields, types and constraints does this event promise?' A schema registry adds shared contract/version management, but compatibility is never just 'old versus new looks similar.' It depends on the serialization format and configured compatibility policy. A practical evolution plan distinguishes additive from destructive change. Adding an optional currency field while older consumers still read `order_id` and amount can often be introduced safely under an appropriate compatibility policy. Renaming `order_id` to `id`, changing amount from numeric to a formatted string, or changing semantic meaning without versioning can break readers even if the producer code runs. During migrations, dual-read/dual-write or versioned events may be safer than a sudden cutover. Schema validity is also not full data quality: a syntactically valid status can still violate business rules. Treat serialization/schema validation as one boundary, then apply domain validation and quarantine/reconciliation as another.

Try it yourself

Design v1 and v2 schemas for a parcel event. Make one backward-compatible additive change and one intentionally breaking change. Explain why.

Fresh changed retry

A new producer needs multiple phone numbers instead of one phone string. Propose a migration that lets old consumers survive while new consumers adopt the richer field.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Kafka moves bytes; schemas create data contracts. Evolve contracts deliberately so producers and consumers can overlap safely during deployment.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-10

Python producer and consumer workflow

Watch first

[1. Confluent - How to use Python with Apache Kafka](#)

Verified chapters 01:09-20:35: clients, use cases, librdkafka, schemas, AdminClient

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 10 of 12 | Primary practice: `practice/kafka/10_*` and simulator evidence where supplied

Why this matters

This lesson turns concepts into the Python client workflow a junior data engineer can actually run: connect, produce with delivery evidence, consume with explicit group/offset behavior, validate, persist, commit and close cleanly.

Picture it in your head

The Python producer is an asynchronous shipping desk: `produce()` queues work and delivery callbacks confirm the broker outcome. The consumer is a loop that repeatedly polls for records and group events. Neither is safe if your code ignores errors and lifecycle cleanup.

Words before we use them

Word	Meaning before we use it
confluent-kafka	High-performance Python client built on librdkafka.
bootstrap.servers	Initial broker addresses used to discover the cluster.
delivery callback	Producer completion/error callback served through poll/flush activity.
group.id	Consumer application identity.
auto.offset.reset	Starting policy when no valid committed offset exists.
close/flush	Lifecycle calls that finish callbacks/leave groups cleanly.

Useful video / source route

[VIDEO: Confluent - How to use Python with Apache Kafka](#) Verified chapters 01:09-20:35: clients, use cases, librdkafka, schemas, AdminClient

[VIDEO: Confluent Developer - The Python Producer Class](#) Optional visual - no verified bounded range; 8-minute Producer lesson

[VIDEO: Confluent Developer - The Python Consumer Class](#) Optional visual - no verified bounded range; 7-minute Consumer lesson

[OFFICIAL/CURRENT: Confluent Python client API 2.15.0](#) Current Producer/Consumer Python API reference

[OFFICIAL/CURRENT: Confluent Developer - Apache Kafka and Python getting started](#) Step-by-step Python client application route

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Run the supplied producer with three shipment events and a delivery callback. Then run the consumer with `group.id=m14-shipment-a`, `auto.offset.reset=earliest` and `enable.auto.commit=False`. Validate JSON, write event IDs to a local idempotency ledger, and commit only after the durable outcome. Stop with Ctrl-C so `close()` leaves the group cleanly.

```
from confluent_kafka import Producer

def delivered(err, msg):
    if err:
        print('FAILED', err)
    else:
        print(msg.topic(), msg.partition(), msg.offset())

p = Producer({'bootstrap.servers': 'localhost:9092'})
p.produce('m14-orders', key='017', value='{"event_id": "E17", "status": "PAID"}', on_delivery=delivered)
p.flush()
```

What you should see

Producer output should report delivered topic/partition/offset. Consumer output should show decoded key/value plus partition/offset, and a second run of the same group should resume after committed work. A fresh group can replay retained records according to its reset policy.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Calling `flush` after every single event makes an asynchronous producer behave synchronously and can destroy throughput; never flushing before process exit can leave queued events unsent. On the consumer side, a busy loop with no error handling can hide broker/client problems.

Reason through a realistic incident

The script prints 'produced' before the callback reports an error. That is why enqueueing locally is not delivery confirmation. Base evidence on callbacks/flush results and broker-side consumption, not a print immediately after `produce()`.

Deepen the engineering reasoning

The Python client workflow should make asynchronous behavior visible. `Producer.produce()` generally enqueues work into the client; delivery callbacks are driven as the client services events, and `flush()` is a shutdown/learning convenience rather than a throughput pattern to call after every record. On the consumer side, `poll()` participates in both record delivery and group/client lifecycle. The supplied route disables auto commit so the durability boundary is explicit. Each processed record should save enough evidence to explain a restart: `event_id`, topic, partition, offset, durable sink outcome and commit outcome. Invalid input should not disappear into console text. Persist a quarantine record with raw payload and coordinates; once that quarantine outcome is durably written, the lesson policy can treat the poison record as handled and commit past it. A production system might instead retry with backoff, send to a dedicated DLQ topic, or stop the partition. The important point is that the policy is explicit, observable and replayable, not accidental behavior hidden inside an exception handler.

Try it yourself

Modify the producer to send five order events with `order_id` keys and modify the consumer group name. Save delivery evidence, consumed partition/offsets and the final ledger count.

Fresh changed retry

Inject one malformed JSON event. The consumer must quarantine it with raw bytes/error context and continue according to an explicit policy rather than crashing forever or silently committing as valid.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Python Kafka clients are asynchronous/networked systems. Observe callbacks/errors, control group identity and progress, and make external processing replay-safe.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

DE14-11

Spark Structured Streaming + Kafka

Watch first

[1. DataBeli - End-to-End Spark Streaming Project](#)

Verified chapters 32:29-57:42: Kafka connection and Spark Kafka consumption; vendor UI differs from local route

[2. Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\)](#)

Verified chapters 00:00-04:10: stream processing and batch/stream comparison

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 11 of 12 | Primary practice: `practice/kafka/11_*` and simulator evidence where supplied

Why this matters

M12 taught Spark and M14 taught Kafka. This lesson joins them without pretending Kafka and Spark are the same system. Kafka is the durable event log; Spark Structured Streaming is a compute engine that reads partitions, tracks query progress/checkpoints and incrementally transforms data.

Picture it in your head

Kafka is the railway carrying numbered carriages. Spark Structured Streaming is a long-running sorting facility beside the track. It repeatedly reads a range of Kafka offsets, transforms records, and records its own checkpoint so the query can recover.

Words before we use them

Word	Meaning before we use it
<code>readStream</code>	Spark API for defining a streaming input.
Kafka source	Structured Streaming connector that exposes Kafka <code>key/value/topic/partition/offset/timestamp</code> columns.
<code>checkpointLocation</code>	Durable Spark query state/progress directory for recovery.
<code>startingOffsets</code>	Spark source option for initial start when a new query has no checkpoint.
micro-batch	Default Structured Streaming execution mode processing offset ranges in small batches.
output mode	Rules for what result rows are emitted as streaming state changes.

Useful video / source route

[OFFICIAL/CURRENT: Official Spark 4.2 Structured Streaming + Kafka integration](#) Current Kafka source/sink, options and deployment package

[VIDEO: DataBeli - End-to-End Spark Streaming Project](#) Verified chapters 32:29-57:42: Kafka connection and Spark Kafka consumption; vendor UI differs from local route

[VIDEO: Confluent Developer - Stream Processing | Apache Kafka 101 \(2025 Edition\)](#) Verified chapters 00:00-04:10: stream processing and batch/stream comparison

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Launch Spark 4.2 with the spark-sql-kafka connector package, subscribe to m14-orders, cast key/value from bytes to strings, parse JSON, filter invalid amounts and write a console sink with a checkpoint directory. Inspect topic, partition and offset columns. Stop/restart with the same checkpoint and observe that the query continues from checkpointed progress rather than blindly applying startingOffsets again.

```
spark-submit \
  --packages org.apache.spark:spark-sql-kafka-0-10_2.13:4.2.0 \
  practice/kafka/11_spark_structured_streaming.py
```

What you should see

The streaming DataFrame schema includes Kafka metadata. Valid events appear after parsing. The checkpoint directory is created after the query starts. Restart behavior follows checkpoint state when it exists.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Do not confuse a Kafka consumer group's committed offsets with Spark Structured Streaming checkpoint semantics. Spark's Kafka source manages its query progress through Spark checkpointing; the integration docs explicitly describe Kafka-specific configuration boundaries.

Reason through a realistic incident

An engineer deletes the Spark checkpoint to 'fix' a stuck query and the pipeline reprocesses old offsets into a non-idempotent sink. The checkpoint was part of correctness, not disposable cache. Recovery should diagnose source offsets, query state and sink idempotency before destructive reset.

Deepen the engineering reasoning

Spark Structured Streaming adds another state/progress layer on top of Kafka. Kafka still owns topic partitions and offsets. Spark uses those offsets as source progress while its checkpoint directory persists query metadata and state needed for recovery. Therefore startingOffsets is mainly a starting rule for a new query without usable prior checkpoint state; an established query normally resumes from checkpoint information. Deleting the checkpoint is not 'clear cache.' It changes the recovery identity and can cause rereads, lost state or different semantics depending on the query and sink. The checkpoint must live on durable storage appropriate to the environment, not a disposable path if the query is production-critical. Kafka partition count also influences source parallelism, while Spark can add downstream shuffle partitions for transformations. Keep those two kinds of partitioning conceptually separate. In this course the console sink is learning evidence only. For durable targets, design idempotent writes and save query progress, source offsets and output counts so restart behavior can be demonstrated rather than guessed.

Try it yourself

Write a Structured Streaming design for parcel events: Kafka source -> parse -> validate -> aggregate by status -> Delta sink. State the source key, checkpoint location and replay-safe sink assumption.

Fresh changed retry

Change the design so malformed events go to a quarantine Delta table and valid events continue. Explain how you would reconcile input counts against valid + quarantine counts per micro-batch/time window.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Kafka retains events; Spark Structured Streaming performs incremental computation and checkpoints its own query state. Treat checkpoints and sink semantics as correctness controls.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.

Why tolerant conversion matters in Spark 4

ANSI behavior can make a normal cast of `abc` or `NaN` into a decimal raise an exception before quarantine can be written. The supplied classifier uses `try_cast` to obtain null for an invalid number, then assigns a reason. It also rejects negative amounts, excessive precision and unknown status. `12.50 / PAID` is valid; `abc` is nonnumeric; `-1` is negative; `1.234` violates the two-decimal contract. Decimal conversion alone could round `1.234` to `1.23`, so the format check remains necessary. The two output writes in `foreachBatch` are not one transaction. The retry replaces only that micro-batch's own paths. Read them only after the query batch succeeds. Never delete the checkpoint while reusing those output directories: the batch IDs can restart. This teaching sink does not demonstrate a general exactly-once multi-table transaction.

DE14-12

Lag, failures and streaming operations

Watch first

[1. Confluent - Slow Consumers monitoring demo](#)

Optional visual - no verified bounded range; older UI but useful lag symptom visual

Pause to connect the demonstration to the example below. Use the printed stop time when a bounded range is given; longer recordings remain available for reference.

M14 lesson 12 of 12 | Primary practice: `practice/kafka/12_*` and simulator evidence where supplied

Why this matters

A streaming pipeline is not finished when messages flow once. Operations means knowing whether it is keeping up, whether one partition is unhealthy, whether rebalances are normal, and how to recover without hiding loss or duplication.

Picture it in your head

Producer progress is the front of a moving queue; consumer committed progress is a runner behind it. Lag is the distance. A little distance can be normal. A distance that grows continuously means the consumer cannot keep pace, is blocked, or is failing/rebalancing.

Words before we use them

Word	Meaning before we use it
consumer lag	Difference between the log end position and a consumer group's committed/processed position, evaluated per partition.
backlog	Accumulated unprocessed events.
rebalance churn	Frequent assignment changes that reduce useful processing time.
DLQ/quarantine	Separate durable path for records that cannot currently be processed safely.
throughput mismatch	Input arrives faster than the consumer application can durably process.
runbook	Documented diagnostic/recovery procedure with evidence and safe actions.

Useful video / source route

[OFFICIAL/CURRENT: Confluent Developer - Kafka Monitoring FAQ](#) Consumer lag definition and JMX/open-source monitoring routes

[VIDEO: Confluent - Slow Consumers monitoring demo](#) Optional visual - no verified bounded range; older UI but useful lag symptom visual

[VIDEO: Confluent Developer - Running Apache Kafka in Production](#) Optional visual - no verified bounded range; monitoring, failure and retry/DLQ reasoning

[VIDEO: Confluent Developer - Consumer Group Protocol](#) Optional visual - no verified bounded range; deeper coordinator/rebalance explanation

Use rule: videos are strongly encouraged for visual learning, but the learner book + supplied practice are self-sufficient. If a video is older or vendor-specific, use it for the concept/visual and use current official docs for version-sensitive commands.

Follow with me once

Use the supplied lag simulator with four partitions. First input rate=40 events/s and processing=50: backlog shrinks. Then make partition 2 process only 2 events/s while the others process 15: overall worker count may look healthy, but partition 2 lag grows. The correct action depends on cause - hot key, poison records, slow external sink, CPU, repeated rebalances - not simply 'restart Kafka'.

What you should see

You should be able to read a per-partition lag table, identify whether lag is uniform or skewed, and choose the next evidence to collect: input rate, processing latency, error rate, rebalance logs, partition/key distribution and sink health.

Why it worked

The result follows from the contract just introduced rather than from a memorized command. Preserve the evidence that proves the contract: keys/partitions/offsets, committed positions, sink reconciliation, checkpoint state or lag by partition as appropriate.

Break it / common failure

Lag=0 is not automatically healthy if no events are arriving, and nonzero lag is not automatically an incident. Alerts need business freshness SLOs and trend/rate context. Another failure is sending poison records to a DLQ and never operating the DLQ.

Reason through a realistic incident

Lag spikes after a deployment. Errors are low but rebalances occur every minute. Check whether consumers are missing heartbeat/poll requirements because processing blocks too long. A scaling change can make things worse if it increases rebalance churn without adding useful partition parallelism.

Deepen the engineering reasoning

Lag is a symptom, not a diagnosis. Compute it per partition and look at the shape. One partition falling behind while others are healthy suggests skew, a hot key, a poison or expensive record, partition-specific broker/network trouble, or sink behavior tied to that data. All partitions rising together after a deployment suggests a broader processing or downstream regression. Frequent rebalances add another clue: consumers may be exceeding poll/processing expectations, crashing, or changing membership too often. Build an incident sequence: confirm producer/log-end movement, measure committed positions and lag, inspect assignments/rebalances, measure processing latency, inspect sink latency/errors, then change one thing. Scale consumers only if partitions and downstream capacity can use the extra workers. Validate recovery by watching lag trend return toward a stable level while business reconciliation remains correct. A consumer catching up quickly but dropping records incorrectly is not a successful recovery. Operations closes the loop between Kafka metrics and the data-quality/reconciliation habits from earlier modules.

Try it yourself

Write a five-step runbook for rising consumer lag. Include at least one metric, one log/evidence check, one no-data-loss action and one validation after recovery.

Fresh changed retry

Now only one partition lags. Rewrite the runbook so it prioritizes skew/hot keys and poison records before cluster-wide scaling.

Validation and recovery ladder

- First prove the broker/source is reachable before changing application logic.
- Name the topic, partition, key/group/query identity and the exact progress marker you are relying on.
- Reconcile input evidence against valid output + quarantine/error outcomes; do not use a green process as proof of correctness.
- If a failure can cause replay, prove the sink or side effect handles replay safely before you call the pipeline reliable.
- Keep the genuine first attempt/error. A corrected run is repair evidence; the changed retry proves mastery.

Revision in your own words

Operate streaming systems per partition and per business freshness objective. Lag is a symptom that needs context, not a diagnosis by itself.

Evidence to keep

Save: command or code used, one expected output, one failure/recovery note, and one sentence explaining the guarantee/boundary in your own words. You are ready only when you can solve the changed retry with the review closed.